

Project Reset Report

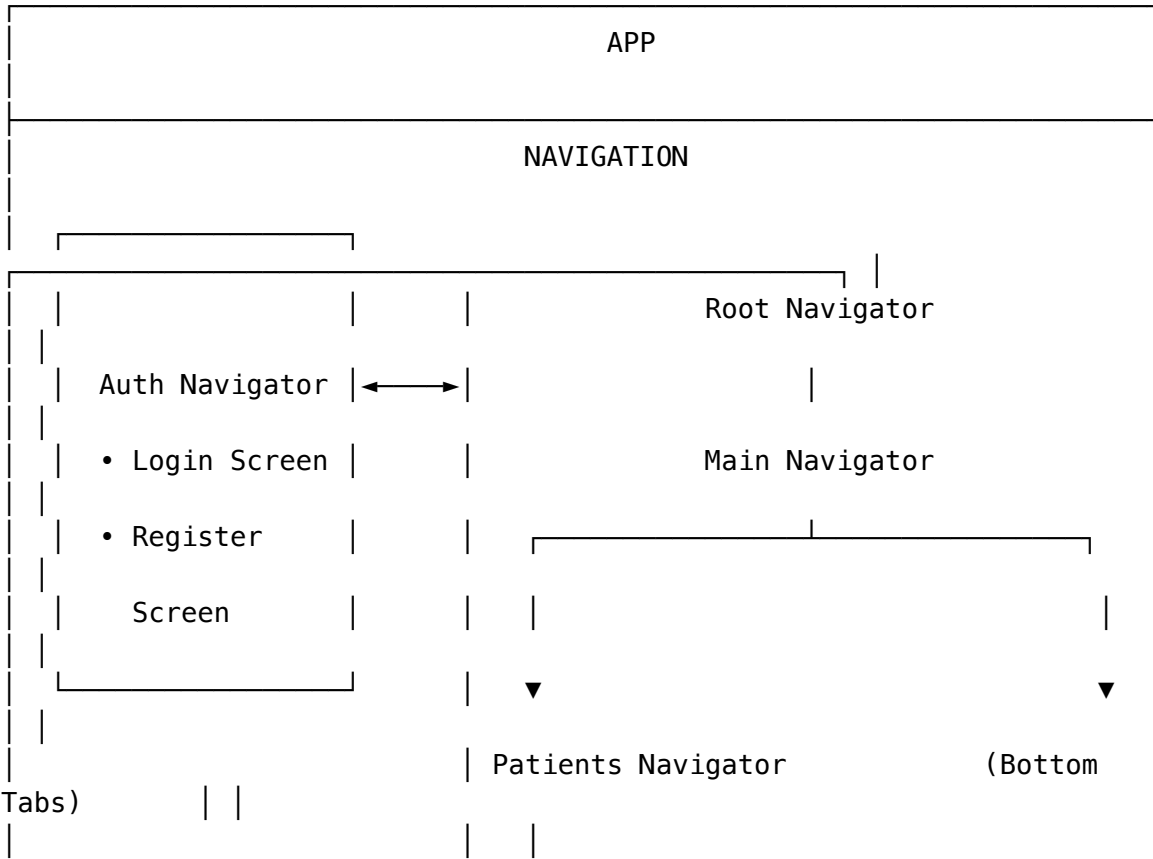
Synka MVP - Project Reset Report

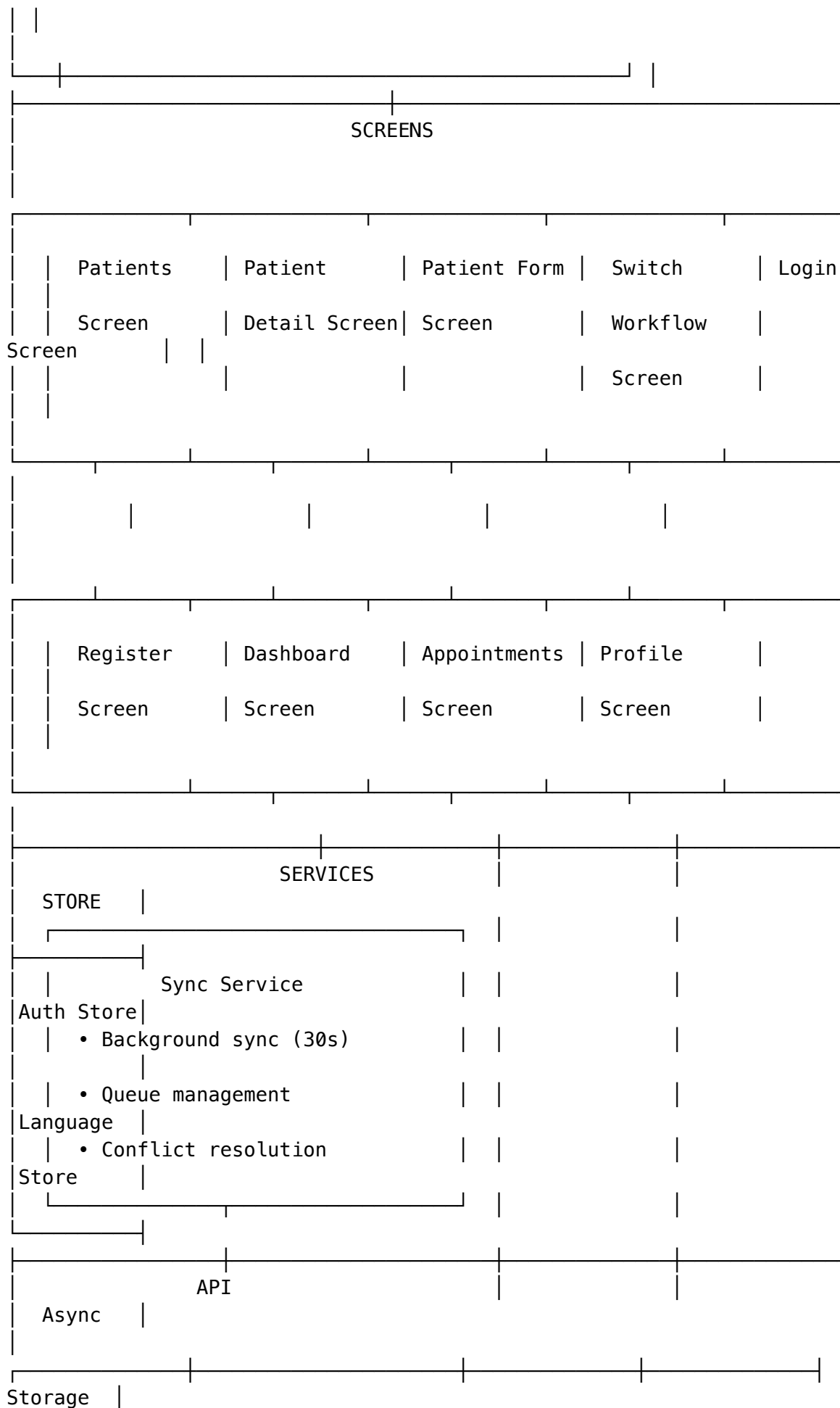
Senior Project II Readiness Assessment

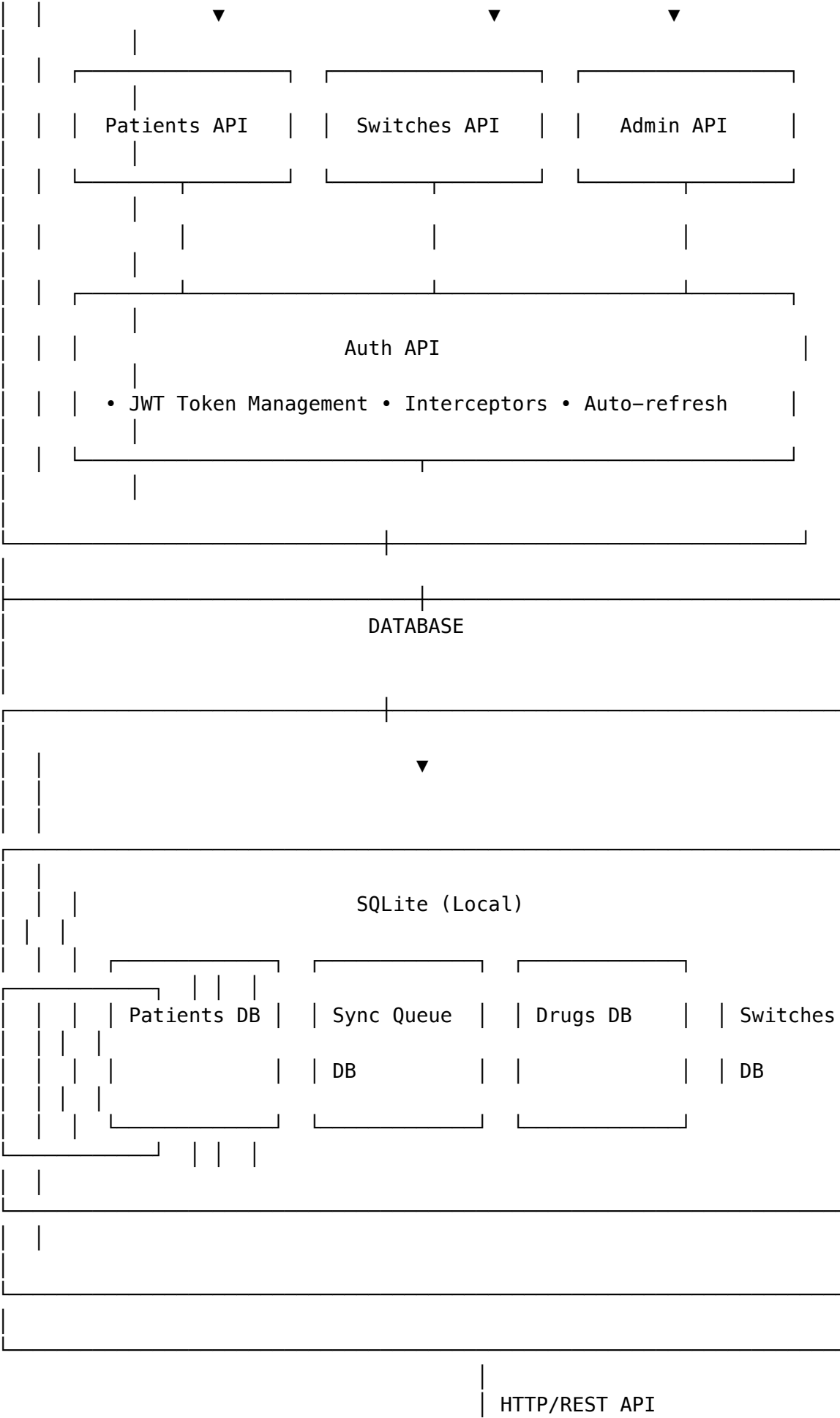
Project: Synka - Biosimilar Switch Kit MVP
Team: Howard University Senior Project Team
Date: February 3, 2026
Assessment Period: Senior Project I (Fall 2024 - Winter 2025)

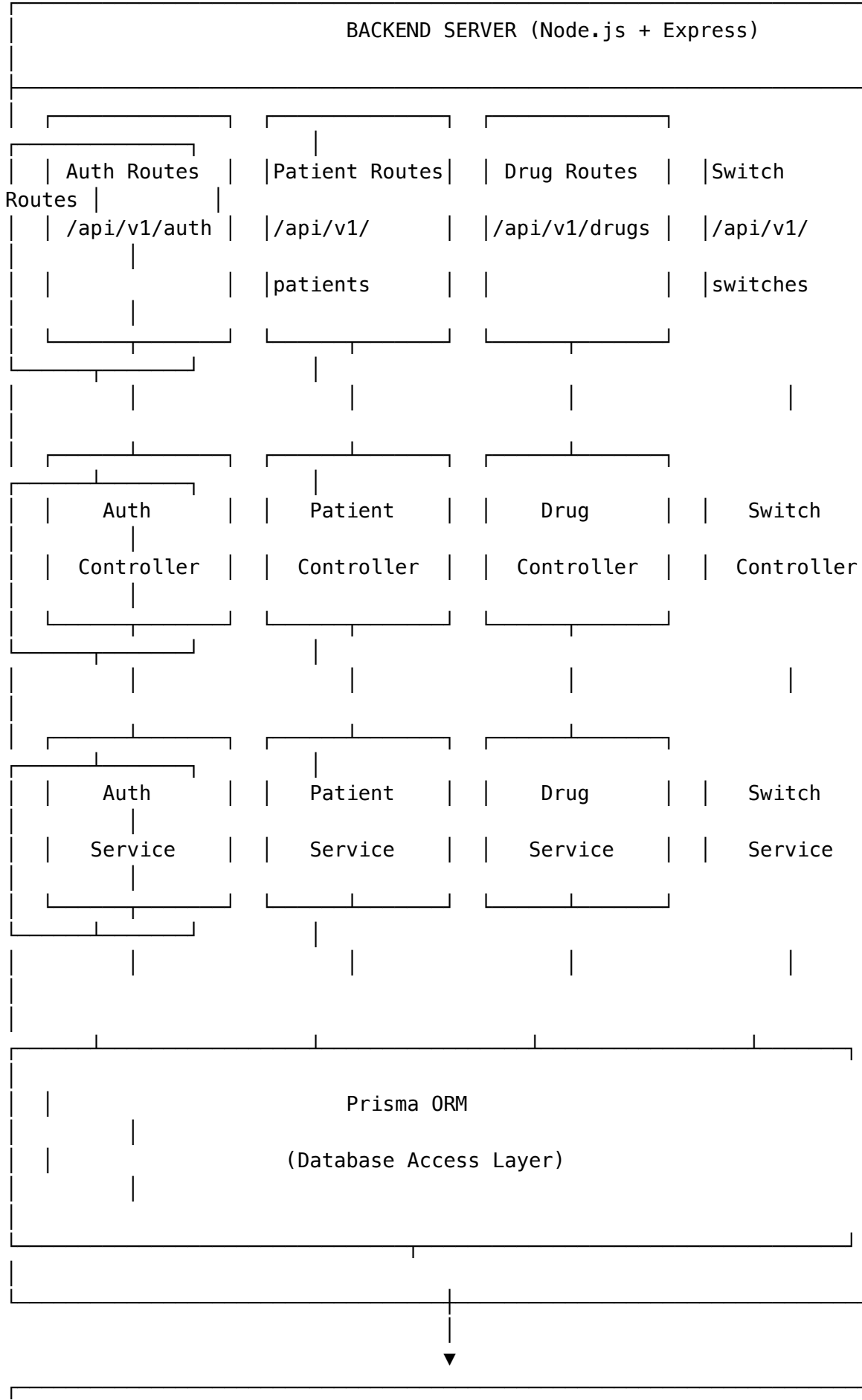
1. Architecture Diagram

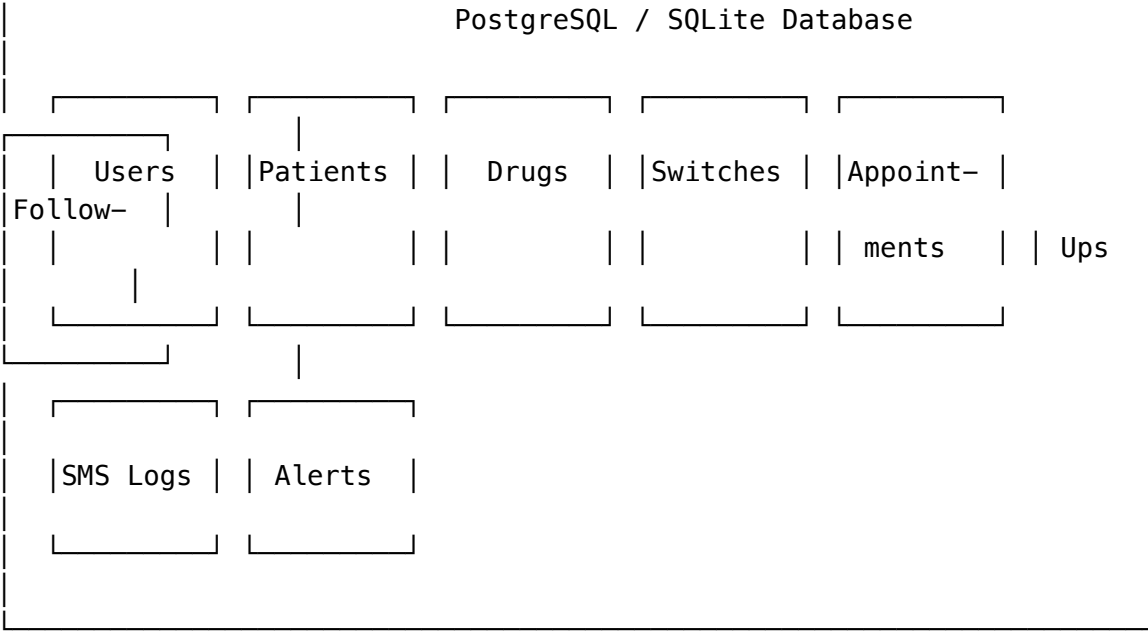
1.1 System Architecture Overview



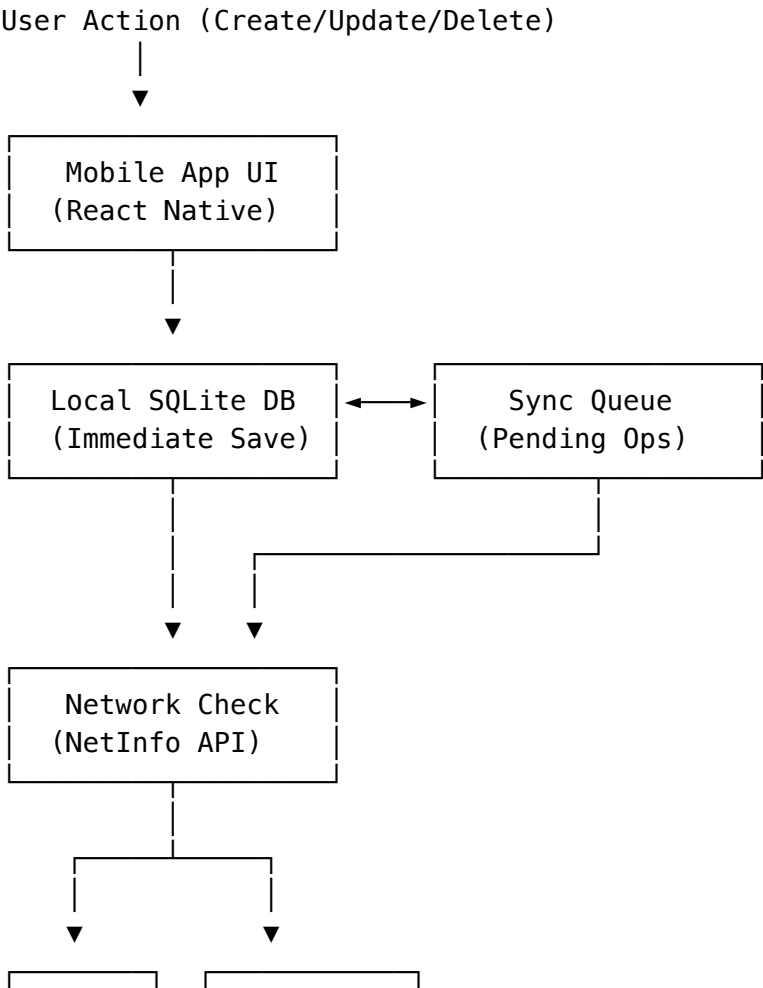
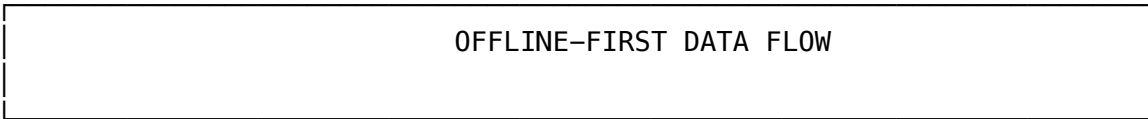


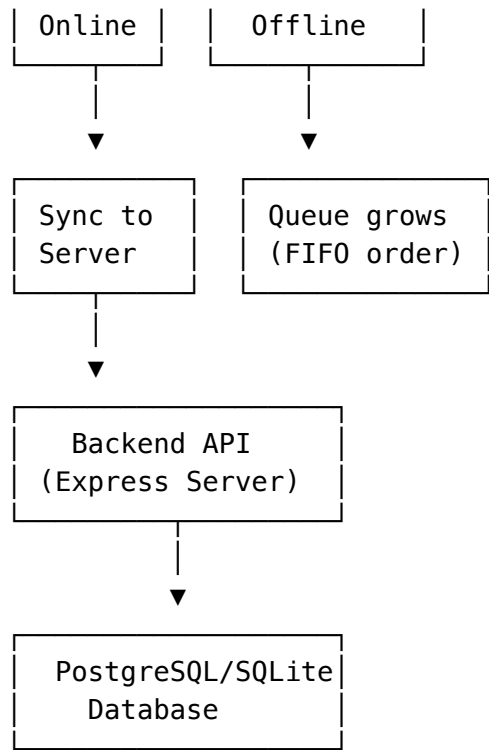




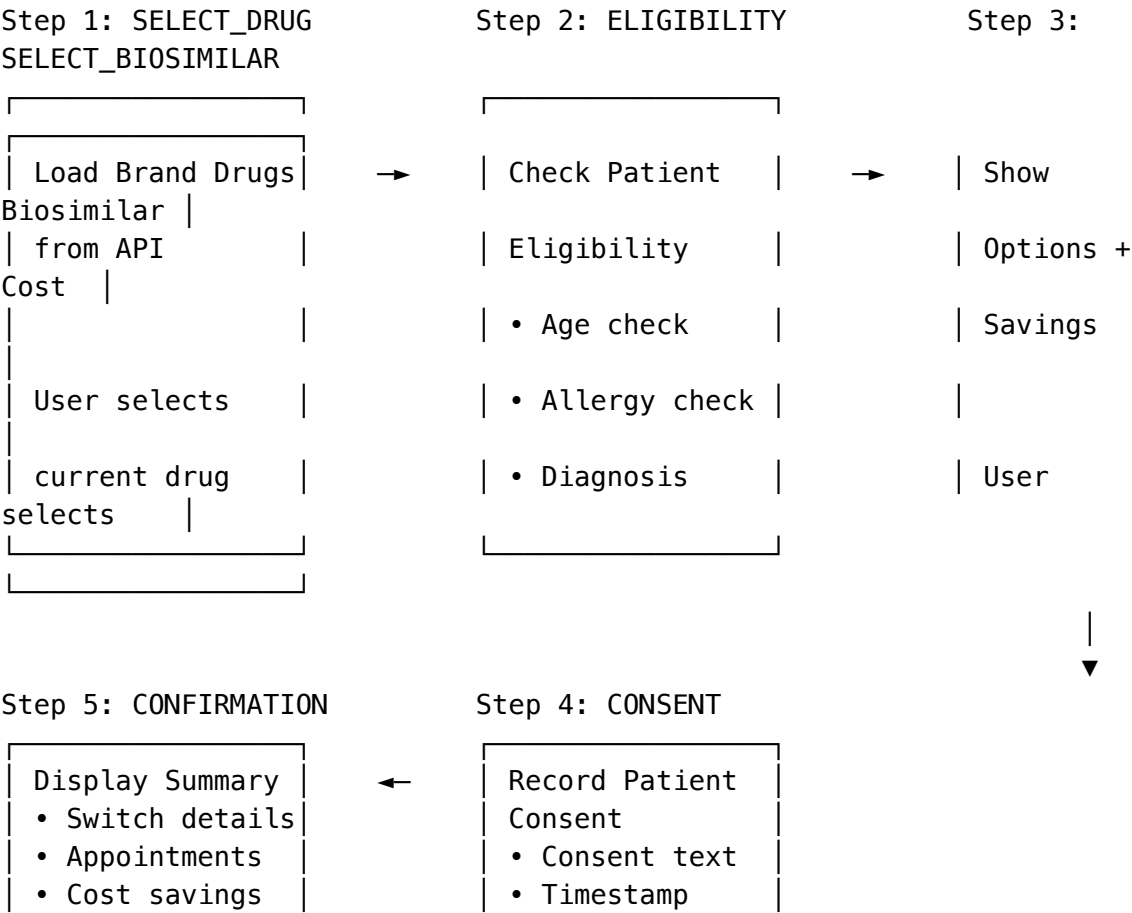


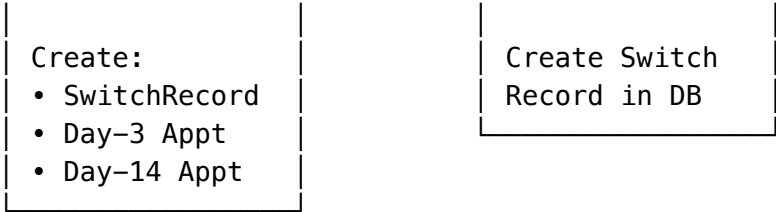
1.2 Data Flow Diagram





BIOSIMILAR SWITCH WORKFLOW





2. Architecture Explanation

2.1 System Overview

Synka is a **mobile-first, offline-capable** healthcare application designed to manage biosimilar medication switches in emerging markets. The architecture follows a **three-tier model**:

Presentation Layer (Mobile App)

- **Framework:** React Native 0.82.1 with TypeScript
- **State Management:** Zustand for auth/language, React Query for server state
- **Navigation:** React Navigation with nested stack and tab navigators
- **UI Library:** React Native Paper (Material Design)

Application Layer (Backend API)

- **Framework:** Node.js 18 + Express.js with TypeScript
- **Authentication:** JWT-based with 7-day token expiration
- **Validation:** Formik + Yup (client), custom middleware (server)
- **ORM:** Prisma 5.x for database operations

Data Layer

- **Server Database:** SQLite (development) / PostgreSQL (production ready)
- **Mobile Database:** react-native-sqlite-storage for offline persistence
- **8 Database Tables:** users, patients, drugs, switch_records, appointments, follow_ups, sms_logs, alerts

2.2 Key Architectural Patterns

Offline-First Architecture

The app implements a robust offline-first pattern: 1. All CRUD operations save to local SQLite immediately 2. Changes queue in sync_queue table when offline 3. Background sync service polls every 30 seconds 4. Server-wins conflict resolution strategy 5. Visual indicators show sync status to users

Component-Based UI Structure

Navigation Layer

- └─ RootNavigator (Auth conditional)
- └─ AuthNavigator (Login, Register)
- └─ MainNavigator (Bottom tabs)
- └─ PatientsNavigator (Stack)

Screen Layer

- └─ Patients (List, Detail, Form)
- └─ Switches (Workflow – 5 steps)
- └─ Appointments (List)
- └─ Dashboard (Metrics)
- └─ Profile (Settings)

Service Layer Pattern

API Layer (Axios)

- └─ Auth API (register, login, me)
- └─ Patients API (CRUD + search)
- └─ Drugs API (list, biosimilars)
- └─ Switches API (workflow, eligibility)
- └─ Admin API (dashboard metrics)

2.3 Technology Stack Summary

Layer	Technology	Purpose
Mobile	React Native 0.82.1	Cross-platform app
Language	TypeScript 5.8	Type safety
Navigation	React Navigation 7.x	Screen routing
State	Zustand 5.x	Client state
Server State	React Query 5.x	Caching, sync
Forms	Formik + Yup	Validation
Local DB	SQLite	Offline storage
Backend	Express.js	REST API
ORM	Prisma 5.x	DB abstraction
Auth	JWT + bcrypt	Security

3. Technical Debt & Risk Identification

3.1 Technical Debt

ID	Debt Item	Severity	Impact	Location
TD-1	SMS Integration Not Implemented	HIGH	Patients won't receive appointment reminders	backend/src/services/
TD-2	SQLite in Production Backend	MEDIUM	Limited scalability, no multi-user support	backend/prisma/schema.prisma
TD-3	No Automated Tests	HIGH	Regression risk, deployment fear	mobile/SynkaApp/__tests__/
TD-4	Hardcoded API URL	LOW	Deployment issues	mobile/SynkaApp/src/constant
TD-5	Missing Error Boundaries	MEDIUM	App crashes on unhandled errors	mobile/SynkaApp/App.tsx
TD-6	Incomplete Localization	MEDIUM	Spanish users see mixed languages	mobile/SynkaApp/src/locales/
TD-7	No API Rate Limiting	MEDIUM	DoS vulnerability	backend/src/middleware/
TD-8	Missing Input Sanitization	HIGH	XSS/Injection risk	backend/src/controllers/
TD-9	Sync Conflicts Not Logged	LOW	Data issues undetectable	mobile/SynkaApp/src/services
TD-10	No Database Encryption	MEDIUM	PHI exposure risk	mobile/SynkaApp/src/database

3.2 Risk Assessment

ID	Risk	Likelihood	Impact	Severity	Mitigation Strategy
R-1	SMS costs exceed budget	Medium	Medium	MEDIUM	Use Twilio trial credits (500 SMS free), limit pilot size, implement rate limiting
R-2	Offline sync data loss	Low	High	MEDIUM	Extensive testing of sync queue, add conflict logging, implement manual conflict resolution UI
R-3	12-week timeline insufficient	High	High	HIGH	Prioritize core switch flow, defer nice-to-haves (charts, exports), MVP-first approach
R-4	Team React Native inexperience	Medium	High	HIGH	Pair programming, code reviews, use established patterns, consult documentation
R-5	Backend downtime during demo	Low	Critical	MEDIUM	Deploy 1 week early, load test, prepare offline demo fallback
R-6	HIPAA compliance gaps	Medium	Critical	HIGH	Audit data handling, implement encryption, document security measures
R-7	Drug data accuracy	Medium	High	HIGH	Cross-reference FDA Purple Book, implement data validation
R-8	Multi-device sync conflicts	Medium	Medium	MEDIUM	Server-wins strategy implemented, add manual resolution for edge cases

3.3 Known Issues

Issue	Status	Priority	Notes
Simulator stuck on Apple logo	Resolved	-	Caused by disk space; freed 6.5GB
Orphaned patient detection incomplete	Open	P2	Need to handle edge cases
Follow-up form doesn't validate	Open	P1	Missing satisfaction

Issue	Status	Priority	Notes
day-14 specific fields	Open	P3	requirement
Dashboard metrics don't update in real-time			Uses pull-to-refresh only

4. Backlog Health Assessment

4.1 Current Backlog Status

Based on review of docs/GITHUB_PROJECT_BACKLOG_SETUP.md:

Column	Item Count	Status
Done	30 items	 Sprints 1-5 complete
Sprint 6	9 items	 Current sprint
Backlog	5 items	 Future features (v2)
In Progress	0	 Not started
Review	0	 Nothing pending

4.2 Sprint Completion Analysis

Sprint	Focus	Items	Status	Notes
Sprint 1	Foundation	6	 100%	Backend + Mobile setup complete
Sprint 2	Patient Management	6	 100%	CRUD + offline sync working
Sprint 3	Switch Workflow	6	 100%	5-step workflow implemented
Sprint 4	Appointments	5	 100%	Auto-scheduling, follow-up forms
Sprint 5	Dashboard & Profile	7	 100%	Metrics, alerts, settings
Sprint 6	SMS & Polish	9	 0%	Not started

4.3 Backlog Quality Assessment

Strengths

-  Clear item naming convention ([EPIC], [US-X.X], [TASK-X.X])
-  Well-defined acceptance criteria per item
-  Story points assigned to tasks
-  Sprint assignments clear
-  Epics properly group related work

Areas for Improvement

Issue	Recommendation
No bug tracking items	Add “Bug” item type for defect tracking
Missing priority on all items	Add P0-P3 priority labels consistently
No blocked/dependencies shown	Add dependency links between items
Sprint 6 items have no assignees	Assign owners before sprint start
Missing “Tech Debt” label	Add technical debt tracking

4.4 Backlog Recommendations

1. Add Missing Items:

- Bug: “Follow-up form validation incomplete”
- Task: “Add automated test suite”
- Task: “PostgreSQL migration”
- Task: “Security audit and fixes”

2. Re-prioritize Sprint 6:

- Move “E2E Testing” to P0 (critical for demo)
- Move “Localization” to P2 (can demo in English)
- Add “Database encryption” as P1

3. Create Definition of Done Checklist:

- ☐ Code reviewed
- ☐ Unit tests passing
- ☐ Works offline
- ☐ Works in both languages
- ☐ Documentation updated

5. Senior Project II Priorities

5.1 Immediate Priorities (Sprint 6 - Current)

Priority	Item	Effort	Owner	Rationale
P0	Complete E2E Testing	8 pts	All	Critical for demo confidence
P0	Bug Fixes from Testing	8 pts	All	Must be stable for presentation
P0	Demo Preparation	5 pts	Simon	Required deliverable
P1	Twilio SMS Integration	8 pts	Cameron	Core feature per PRD
P1	Automated SMS Reminders	5 pts	Basanta	Patient engagement
P1	UI Polish & Error Handling	5 pts	Sollomon/Destin	User experience
P2	Localization Completion	3 pts	Destin	Spanish support
P2	Documentation	3 pts	Simon	Knowledge transfer

5.2 Senior Project II Roadmap

Phase 1: Stabilization (Weeks 1-2)

- Complete Sprint 6 items
- Conduct comprehensive testing
- Fix all critical bugs
- Prepare demo environment

Phase 2: Security & Compliance (Weeks 3-4)

- Implement database encryption (SQLCipher)
- Add input sanitization
- Implement rate limiting
- Document security measures

Phase 3: Production Readiness (Weeks 5-6)

- Migrate to PostgreSQL
- Deploy to cloud (Railway/Render)
- Set up monitoring
- Load testing

Phase 4: Enhancement (Weeks 7-8)

- Advanced reporting/export
- Push notifications
- Performance optimization
- User feedback incorporation

Phase 5: Final Delivery (Weeks 9-10)

- Final testing round
- Documentation completion
- Knowledge transfer
- Final presentation

5.3 Success Criteria for Senior Project II

Metric	Target	Current	Gap
Patients registered	>50	TBD	Need pilot
Complete switch workflows	>5	TBD	Need pilot
SMS delivery rate	>95%	0%	Not implemented
Offline sync success	>99%	~95%	Need testing
App crash rate	<0.1%	Unknown	Need monitoring
Test coverage	>70%	<5%	Major gap
Spanish translations	100%	~60%	40% remaining

6. Appendices

Appendix A: File Structure Reference

```
synka/
├── backend/
│   ├── prisma/
│   │   └── schema.prisma      # Database schema (8 models)
│   └── src/
│       ├── controllers/      # Request handlers (4 files)
│       ├── services/         # Business logic (4 files)
│       ├── routes/           # API endpoints (5 files)
│       └── middleware/       # Auth, validation, errors
├── mobile/SynkaApp/
│   └── src/
│       ├── api/              # API clients (6 files)
│       ├── components/       # Reusable UI (5 files)
│       ├── screens/          # App screens (10 files)
│       ├── navigation/       # Navigators (5 files)
│       ├── database/         # SQLite ops (4 files)
│       └── services/         # Sync service
```

```
|
|   ├── store/           # Zustand stores (3 files)
|   └── hooks/          # Custom hooks (1 file)
└── docs/
    ├── PRD_Synka_MVP.md # Product requirements
    └── GITHUB_PROJECT_BACKLOG_SETUP.md
```

Appendix B: API Endpoints Implemented

Method	Endpoint	Status	Notes
POST	/api/v1/auth/register	✓	User registration
POST	/api/v1/auth/login	✓	JWT authentication
GET	/api/v1/auth/me	✓	Current user
GET	/api/v1/patients	✓	List with search
POST	/api/v1/patients	✓	Create patient
GET	/api/v1/patients/:id	✓	Get patient
PUT	/api/v1/patients/:id	✓	Update patient
DELETE	/api/v1/patients/:id	✓	Delete patient
GET	/api/v1/drugs	✓	List drugs
GET	/api/v1/drugs/:id/biosimilars	✓	Get alternatives
POST	/api/v1/switches	✓	Create switch
GET	/api/v1/switches/eligibility	✓	Check eligibility
POST	/api/v1/switches/:id/consent	✓	Record consent
POST	/api/v1/appointments/:id/follow-up	✓	Record follow-up
GET	/api/v1/admin/dashboard	✓	Dashboard metrics
POST	/api/v1/sms/send	✗	Not implemented

Appendix C: Team Responsibilities

Member	Role	Primary Responsibility
Simon Armstrong	Scrum Leader	Project management, documentation, demo
Cameron Carter	Technical Lead	Architecture, sync service, SMS integration
Basanta Baral	Backend Dev	API development, database, eligibility engine
Sollomon Crowder	Frontend Engineer	Patient screens, dashboard, navigation

Member	Role	Primary Responsibility
Destin Gilbert	Frontend Engineer	Switch workflow, follow-up forms, localization

Report Prepared By: Basanta Baral
Date: February 3, 2026
Version: 1.0

This document serves as the Project Reset Report for Senior Project II, providing a comprehensive assessment of the Synka MVP’s current technical state and readiness for continued development.